

Automatic Writing Evaluation on Regression

MLP LAB



These slides created with reference to ML for Everyone Season2 with Pytorch

Contents

01 Basic concept of ML

02 Linear Regression on Pytorch

03 Multi-variable LR

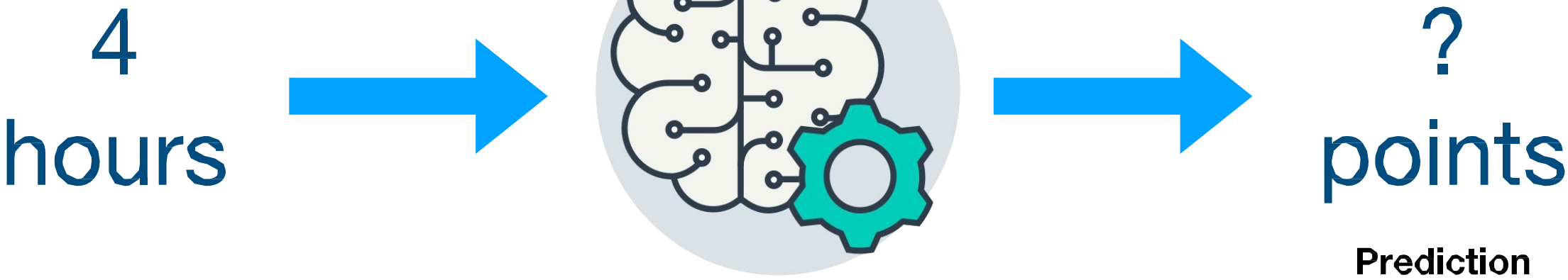
04 Batch based Gradient Descent



01. Concept of ML

Problem definition

- What would be the grade if I study 4 hours?



Hours (x)	Points (y)
1	2
2	4
3	6
4	?

Training dataset

Test dataset

Preparing Dataset

- Let's just convert the data to a Pytorch format

```
x_train = torch.FloatTensor([[1], [2], [3]])  
y_train = torch.FloatTensor([[2], [4], [6]])
```

$$X_{\text{train}} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \quad Y_{\text{train}} = \begin{pmatrix} 2 \\ 4 \\ 6 \end{pmatrix}$$

Hours (x)	Points (y)
1	2
2	4
3	6

The data is torch.tensor.
Inputs and outputs are handled separately.

- Input: x_train
- Output: y_train
- Input and output are separated by rows.

A question

- So how do we make a computer understand real-world problems?

Well, by the looks of it, we can just create a function.

Study Time $\times$ *Weight* = *Score*

A solution: Hypothesis function

Well, by the looks of it, we can just create a function.

Study Time $\times$ *Weight* = *Score*

```
x_train = torch.FloatTensor([[1], [2], [3]])  
y_train = torch.FloatTensor([[2], [4], [6]])
```

```
w = torch.FloatTensor([2])
```

```
def f(x):  
    y = w*x  
    return y
```

```
for a_sample in x_train:  
    prediction = f(a_sample)  
    print(prediction)
```

A solution: Loss Function

How accurately does our function predict?
We can't just rely on a hunch!

```
x_train = torch.FloatTensor([[1], [2], [3]])  
y_train = torch.FloatTensor([[2], [4], [6]])
```

```
w = torch.FloatTensor([2])
```

```
def f(x):  
    y = w*x  
    return y
```

```
def loss(predicted, target):  
    return predicted - target
```

```
loss_value = 0  
for a_sample in x_train:  
    prediction = f(a_sample)  
    loss_value += loss(prediction, y_train)
```


The problem

Uh oh, we've got new training data.

```
x_train = torch.FloatTensor([[1], [2], [3], [1], [3], [8]])  
y_train = torch.FloatTensor([[2], [4], [6], [3], [5], [7]])
```

```
w = torch.FloatTensor([2])
```

← ??

```
def f(x):  
    y = w*x  
    return y
```

```
def loss(predicted, target):  
    return predicted - target
```

```
loss_value = 0  
for a_sample in x_train:  
    prediction = f(a_sample)  
    loss_value += loss(prediction, y_train)
```

Hours (x)	Points (y)
1	2
2	4
3	6
1	3
3	5
8	7

Fundamental Solution: Machine Learning

Can't we just have the computer automatically pick the best **W** value for the data?

```
x_train = torch.FloatTensor([[1], [2], [3], [1], [3], [8]])  
y_train = torch.FloatTensor([[2], [4], [6], [3], [5], [7]])
```

```
W = torch.FloatTensor([2])
```

← ??

```
def f(x):  
    y = w*x  
    return y
```

```
def loss(predicted, target):  
    return predicted - target
```

```
loss_value = 0  
for a_sample in x_train:  
    prediction = f(a_sample)  
    loss_value += loss(prediction, y_train)
```

Hours (x)	Points (y)
1	2
2	4
3	6
1	3
3	5
8	7

Fundamental Solution: Machine Learning

Let's go on a journey to find the optimal **W** based on our observed data.
First, let's take a look at the function we came up with.

$$y = Wx + b$$

Weight

Bias

```
def f(x):  
    y = w*x + b  
    return y
```

```
def loss(predicted, target):  
    return (predicted - target)**2
```

Hours (x)	Points (y)
1	2
2	4
3	6
1	3
3	5
8	7

Supervised Learning with random selection

If we just throw in random W values, the W that makes the loss zero would be ideal, right?
Let's plot the loss value as a function of W . How about $W=2$?

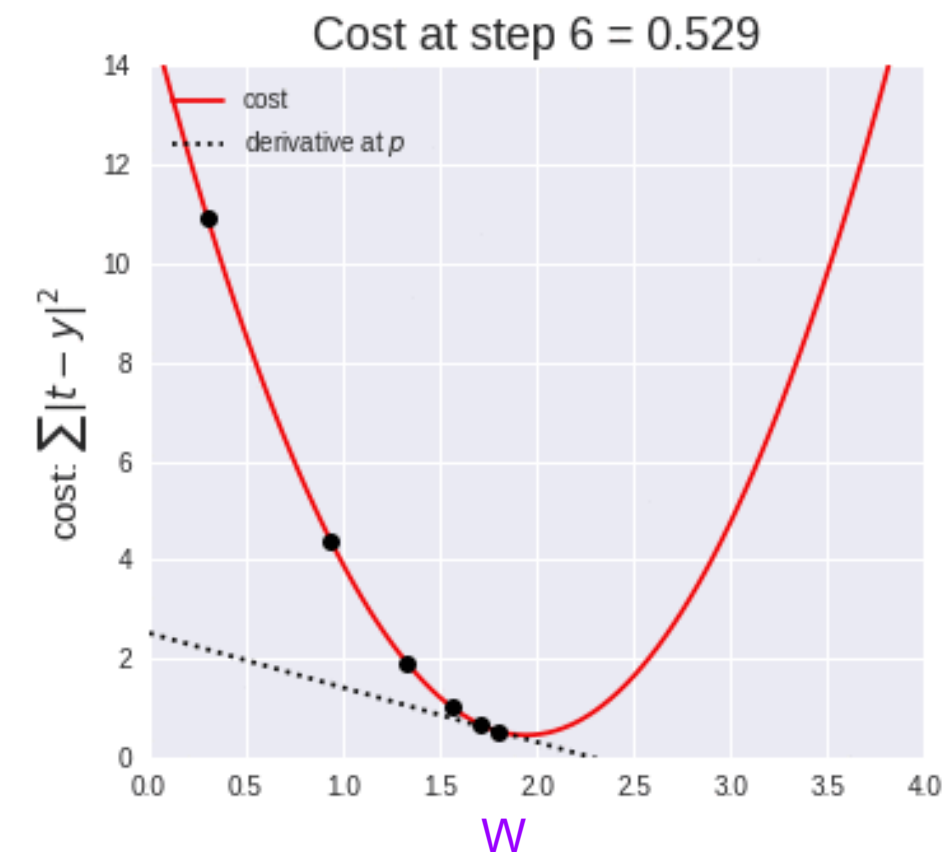
```
w = torch.FloatTensor([2])
```

```
for w_value in range(-100, 100, 0.1):  
    w = torch.FloatTensor([w_value])
```

```
def f(x):  
    y = w*x + b  
    return y
```

```
def loss(predicted, target):  
    return (predicted - target)**2
```

```
loss_v = loss(f(w), y_train)
```



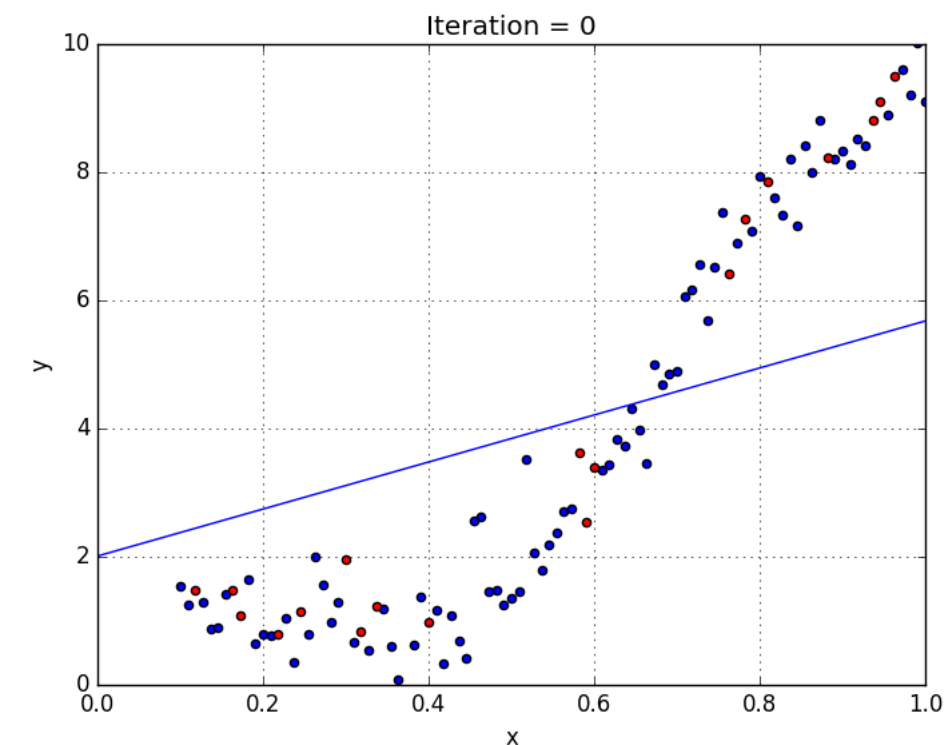
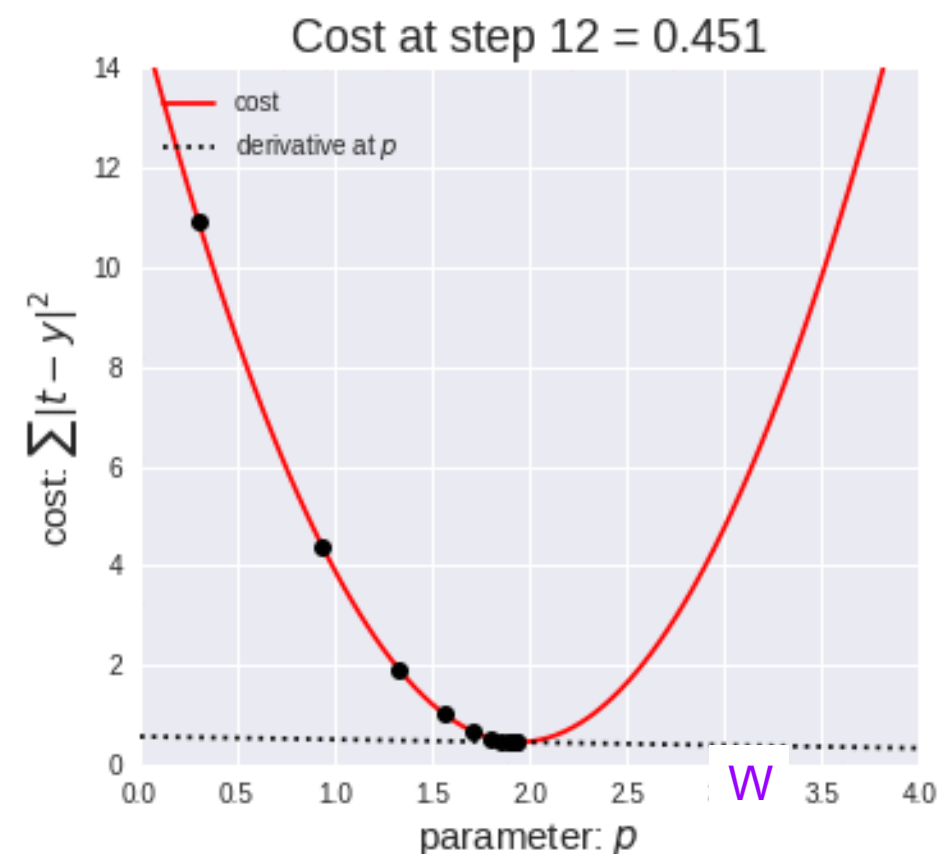
Hours (x)	Points (y)
1	2
2	4
3	6

Gradient Descent

But can we really try out infinitely many random **W** values?

- Instead, let's update **W** based on how it affects the loss value!
- If the gradient is large, we can move further in that direction to descend the curve.

Let's calculate the "Gradient" to do that. $\frac{\partial \text{cost}}{\partial W} = \nabla W$



Gradient Descent

The impact of W on cost = slope = derivative

- If the gradient is large, we can move further in that direction to descend the curve.

Let's calculate the "Gradient" to do that. $\frac{\partial cost}{\partial W} = \nabla W$

$$cost(W) = \frac{1}{m} \sum_{i=1}^m (W x^{(i)} - y^{(i)})^2$$

$$\nabla W = \frac{\partial cost}{\partial W} = \frac{2}{m} \sum_{i=1}^m (W x^{(i)} - y^{(i)}) x^{(i)}$$

$$W : = W - \alpha \nabla W$$

Learning rate Gradient

Gradient Descent

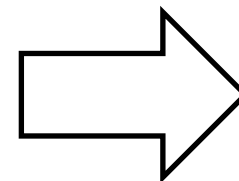
- Let's code it up!

$$\nabla W = \frac{\partial cost}{\partial W} = \frac{2}{m} \sum_{i=1}^m (W x^{(i)} - y^{(i)}) x^{(i)}$$

$$W := W - \alpha \nabla W$$

```
def f(x):  
    y = w*x #+ b  
    return y
```

```
def loss(predicted, target):  
    return (predicted - target)**2
```



```
W = torch.zeros(1, requires_grad=True)  
# b = torch.zeros(1, requires_grad=True)  
hypothesis = x_train * W
```

```
cost = torch.mean((hypothesis - y_train) ** 2)
```

```
gradient = 2 * torch.mean((W * x_train - y_train) * x_train)  
lr = 0.1  
W -= lr * gradient
```

Training

- Let's code it up!

```
# data
x_train = torch.FloatTensor([[1], [2], [3]]) Set for the easiest
y_train = torch.FloatTensor([[1], [2], [3]]) example!
# Model Initialization
W = torch.zeros(1)
# Learning rate
lr = 0.1

nb_epochs = 10
for epoch in range(nb_epochs + 1):

    # H(x)
    hypothesis = x_train * W

    # cost gradient computation
    cost = torch.mean((hypothesis - y_train) ** 2)
    gradient = torch.sum((W * x_train - y_train) * x_train)

    print('Epoch {:4d}/{:} W: {:.3f}, Cost: {:.6f}'.format(
        epoch, nb_epochs, W.item(), cost.item()
    ))

    # cost gradient로 H(x) 개선
    W -= lr * gradient
```

Epoch	0/10	W: 0.000,	Cost: 4.666667
Epoch	1/10	W: 1.400,	Cost: 0.746666
Epoch	2/10	W: 0.840,	Cost: 0.119467
Epoch	3/10	W: 1.064,	Cost: 0.019115
Epoch	4/10	W: 0.974,	Cost: 0.003058
Epoch	5/10	W: 1.010,	Cost: 0.000489
Epoch	6/10	W: 0.996,	Cost: 0.000078
Epoch	7/10	W: 1.002,	Cost: 0.000013
Epoch	8/10	W: 0.999,	Cost: 0.000002
Epoch	9/10	W: 1.000,	Cost: 0.000000
Epoch	10/10	W: 1.000,	Cost: 0.000000

Epoch: The number of times we train on our data

- As we keep training:
 - **W** gradually converges to the correct value
 - The cost keeps decreasing

02. ML using Pytorch

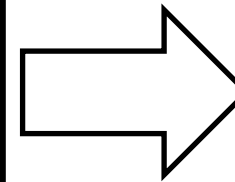
Problems with Implementing Gradient Descent

- If our hypothesis function is a 2D function $f(x)=Wx^2$, we'll need to change the gradient calculation code!
- It would be great if, even when $f(x)$ keeps changing, we could automatically compute the derivative.

Gradient Descent with torch.optim

- Using ***torch.optim*** to Automatically Compute Derivatives and Perform Gradient Descent
 - Define the optimizer at the start.
 - Call ***optimizer.zero_grad()*** to reset gradients to zero.
 - Run ***cost.backward()*** to calculate gradients.
 - Finally, call ***optimizer.step()*** to perform the gradient descent step.

```
gradient = 2 * torch.mean((W * x_train - y_train) * x_train)
lr = 0.1
W -= lr * gradient
```



```
# optimizer settings
optimizer = optim.SGD([W], lr=0.15)

# cost based on H(x)
개선
optimizer.zero_grad()
cost.backward()
optimizer.step()
```

Python VS. Pytorch

```
# 데이터
x_train = torch.FloatTensor([[1], [2], [3]])
y_train = torch.FloatTensor([[1], [2], [3]])
# 모델 초기화
W = torch.zeros(1) # learning
rate 설정 lr = 0.1

nb_epochs = 10
for epoch in range(nb_epochs + 1):

    # H(x) 계산
    hypothesis = x_train * W

    # cost gradient 계산
    cost = torch.mean((hypothesis - y_train) ** 2)
    gradient = torch.sum((W * x_train - y_train) *
                          x_train)

    print('Epoch {:4d}/{:} W: {:.3f}, Cost:
          {:.6f}'.format( epoch, nb_epochs, W.item(),
                          cost.item()))

    # cost gradient로 H(x) 개선
    W -= lr * gradient
```

```
# 데이터
x_train = torch.FloatTensor([[1], [2], [3]])
y_train = torch.FloatTensor([[1], [2], [3]])
# 모델 초기화
W = torch.zeros(1, requires_grad=True)
# optimizer 설정
optimizer = optim.SGD([W], lr=0.15)

nb_epochs = 10
for epoch in range(nb_epochs + 1):

    # H(x) 계산
    hypothesis = x_train * W

    # cost 계산
    cost = torch.mean((hypothesis - y_train) ** 2)

    print('Epoch {:4d}/{:} W: {:.3f} Cost:
          {:.6f}'.format( epoch, nb_epochs, W.item(),
                          cost.item()))

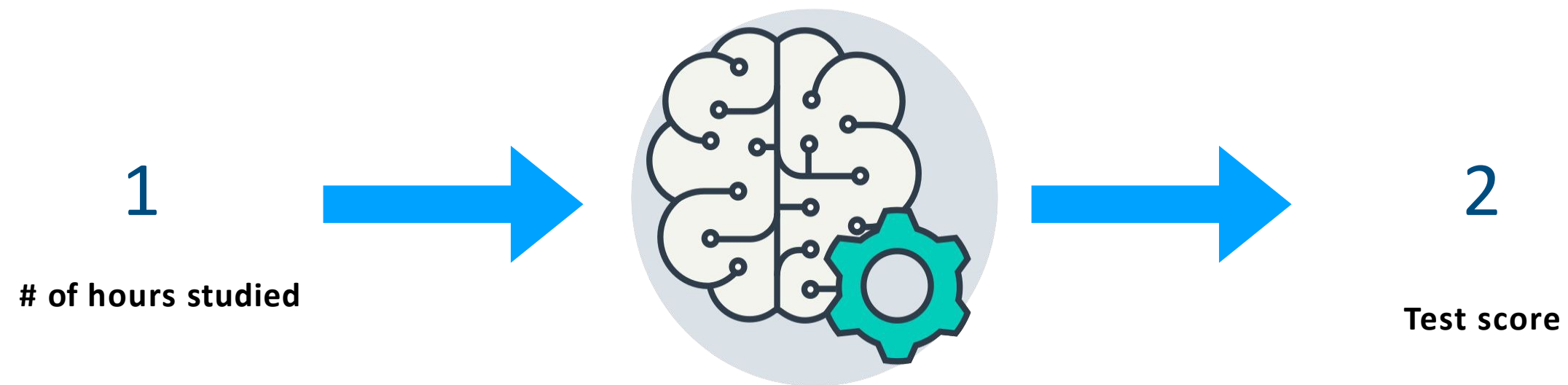
    # cost로 H(x) 개선
    optimizer.zero_grad()
    cost.backward()
    optimizer.step()
```

- Let's Build an Automatic Writing Evaluation (AWE) System
 - 1) Read the data from **AWE_1000_score.csv**
 - 2) Count how many sentences are in the content column, and store that count in a new sentence column.
 - 3) Use the number of sentences as input to a model that predicts the writing evaluation score of each entry.

	id	nationality	sex	title	content	phrase	sentence
0	A100000_v01.txt_1.txt	중국	남자	사진기 빌리기	하지만 빌 리씨하고 나오코씨 는 모두 사 진기가 없 었어요. 그 래서 빌리 씨는 모하 메드 씨...	1.0	5.0

03. Multivariable Linear Model

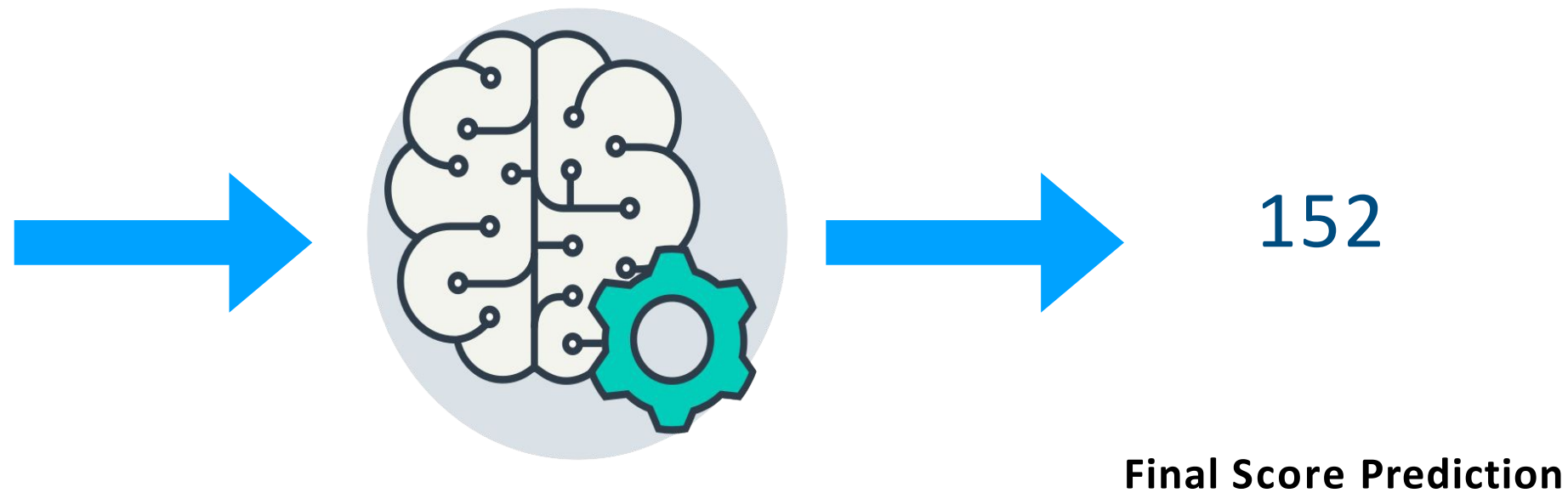
Single Variable Linear Regression



$$H(x) = Wx + b$$

Multi-Variable Linear Regression

Quiz1: 73
Quiz2: 80
Quiz3: 75
Quiz Scores



$$H(x) = Wx + b$$

Multi-Variable Linear Regression

- Dataset

Quiz 1 (x1)	Quiz 2 (x2)	Quiz 3 (x3)	Final (y)
73	80	75	152
93	88	93	185
89	91	80	180
96	98	100	196
73	66	70	142

```
x_train = torch.FloatTensor([[73, 80, 75],  
                             [93, 88, 93],  
                             [89, 91, 90],  
                             [96, 98, 100],  
                             [73, 66, 70]])  
y_train = torch.FloatTensor([[152], [185], [180], [196], [142]])
```

Multi-Variable Linear Regression

- Hypothesis Function

$$H(x) = Wx + b$$

$$H(x) = w_1x_1 + w_2x_2 + w_3x_3 + b$$

If There Are Three Input Variables, Then We Need Three Weights!

$$(x_1 \quad x_2 \quad x_3) \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = (x_1w_1 + x_2w_2 + x_3w_3)$$

$$H(X) = XW$$

Multi-Variable Linear Regression

- Hypothesis Function

```
# H(x) 계산  
hypothesis = x1_train * w1 + x2_train * w2 + x3_train * w3 + b
```

- A Simple Hypothesis Definition!
- But what if the input **x** is a vector of length 1,000?

$$H(x) = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

Multi-Variable Linear Regression

- Understanding Parallel Computation

Quiz 1 (x1)	Quiz 2 (x2)	Quiz 3 (x3)
73	80	75

$$\begin{pmatrix} x_1 & x_2 & x_3 \end{pmatrix} \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = (x_1 w_1 + x_2 w_2 + x_3 w_3)$$

Quiz 1 (x1)	Quiz 2 (x2)	Quiz 3 (x3)
73	80	75
93	88	93
89	91	80
96	98	100
73	66	70

$$\begin{pmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \\ x_{41} & x_{42} & x_{43} \\ x_{51} & x_{52} & x_{53} \end{pmatrix} \cdot \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix} = \begin{pmatrix} x_{11}w_1 + x_{12}w_2 + x_{13}w_3 \\ x_{21}w_1 + x_{22}w_2 + x_{23}w_3 \\ x_{31}w_1 + x_{32}w_2 + x_{33}w_3 \\ x_{41}w_1 + x_{42}w_2 + x_{43}w_3 \\ x_{51}w_1 + x_{52}w_2 + x_{53}w_3 \end{pmatrix}$$

Multi-Variable Linear Regression

- Hypothesis Function

```
# H(x)  
hypothesis = x_train.matmul(W) + b # or .mm or @
```

- Compute Everything at Once with *matmul()*
 - It's more concise.
 - No need to change the code if the length of X changes.
 - It's faster, too!

$$H(x) = Wx + b$$

Multi-Variable Linear Regression

- Cost Function: MSE

$$\text{cost}(W) = \frac{1}{m} \sum_{i=1}^m \left(\underset{\text{Mean}}{\underbrace{\frac{1}{m}}}_{\text{Mean}} \underset{\text{Prediction}}{\underbrace{H(x^{(i)})}_{\text{Prediction}}} - \underset{\text{Target}}{\underbrace{y^{(i)}}_{\text{Target}}} \right)^2$$

```
cost = torch.mean((hypothesis - y_train) ** 2)
```

Multi-Variable Linear Regression

- Cost Function: MSE

$$\nabla W = \frac{\partial cost}{\partial W} = \frac{2}{m} \sum_{i=1}^m (W x^{(i)} - y^{(i)}) x^{(i)}$$

$$W := W - \alpha \nabla W$$

```
# Set optimizer
optimizer = optim.SGD([W, b], lr=1e-5)

# Usage of optimizer
optimizer.zero_grad()
cost.backward()
optimizer.step()
```

Multi-Variable Linear Regression

- Full code with Pytorch

```
# Data
x_train = torch.FloatTensor([[73, 80, 75],[93, 88, 93],
                             [89, 91, 90],
                             [96, 98, 100],
                             [73, 66, 70]])
y_train = torch.FloatTensor([[152], [185], [180], [196], [142]])
# Model Init
W = torch.zeros((3, 1), requires_grad=True)
b = torch.zeros(1, requires_grad=True)
# Setting of optimizer
optimizer = optim.SGD([W, b], lr=1e-5)
```

1. 데이터 정의

2. 모델 정의

3. optimizer 정의

```
nb_epochs = 20
for epoch in range(nb_epochs + 1):

    # H(x)
    hypothesis = x_train.matmul(W) + b # or .mm or @ 4. Hypothesis 계산

    # cost
    cost = torch.mean((hypothesis - y_train) ** 2) 5. Cost 계산 (MSE)

    # H(x) based on the cost
    optimizer.zero_grad()
    cost.backward()
    optimizer.step() 6. Gradient descent

    print('Epoch {:4d}/{:4d} hypothesis: {} Cost: {:.6f}'.format(
        epoch, nb_epochs, hypothesis.squeeze().detach(),
        cost.item()
    ))
```


Multi-Variable Linear Regression

- Full code with Pytorch

```
Epoch 0/20 hypothesis: tensor([0., 0., 0., 0., 0.]) Cost: 29661.800781
Epoch 1/20 hypothesis: tensor([67.2578, 80.8397, 79.6523, 86.7394, 61.6605]) Cost: 9298.520508
Epoch 2/20 hypothesis: tensor([104.9128, 126.0990, 124.2466, 135.3015, 96.1821]) Cost: 2915.713135
Epoch 3/20 hypothesis: tensor([125.9942, 151.4381, 149.2133, 162.4896, 115.5097]) Cost: 915.040527
Epoch 4/20 hypothesis: tensor([137.7968, 165.6247, 163.1911, 177.7112, 126.3307]) Cost: 287.936005
Epoch 5/20 hypothesis: tensor([144.4044, 173.5674, 171.0168, 186.2332, 132.3891]) Cost: 91.371017
Epoch 6/20 hypothesis: tensor([148.1035, 178.0144, 175.3980, 191.0042, 135.7812]) Cost: 29.758139
Epoch 7/20 hypothesis: tensor([150.1744, 180.5042, 177.8508, 193.6753, 137.6805]) Cost: 10.445305
Epoch 8/20 hypothesis: tensor([151.3336, 181.8983, 179.2240, 195.1707, 138.7440]) Cost: 4.391228
Epoch 9/20 hypothesis: tensor([151.9824, 182.6789, 179.9928, 196.0079, 139.3396]) Cost: 2.493135
Epoch 10/20 hypothesis: tensor([152.3454, 183.1161, 180.4231, 196.4765, 139.6732]) Cost: 1.897688
Epoch 11/20 hypothesis: tensor([152.5485, 183.3610, 180.6640, 196.7389, 139.8602]) Cost: 1.710541
Epoch 12/20 hypothesis: tensor([152.6620, 183.4982, 180.7988, 196.8857, 139.9651]) Cost: 1.651413
Epoch 13/20 hypothesis: tensor([152.7253, 183.5752, 180.8742, 196.9678, 140.0240]) Cost: 1.632387
Epoch 14/20 hypothesis: tensor([152.7606, 183.6184, 180.9164, 197.0138, 140.0571]) Cost: 1.625923
Epoch 15/20 hypothesis: tensor([152.7802, 183.6427, 180.9399, 197.0395, 140.0759]) Cost: 1.623412
Epoch 16/20 hypothesis: tensor([152.7909, 183.6565, 180.9530, 197.0538, 140.0865]) Cost: 1.622141
Epoch 17/20 hypothesis: tensor([152.7968, 183.6643, 180.9603, 197.0618, 140.0927]) Cost: 1.621253
Epoch 18/20 hypothesis: tensor([152.7999, 183.6688, 180.9644, 197.0662, 140.0963]) Cost: 1.620500
Epoch 19/20 hypothesis: tensor([152.8014, 183.6715, 180.9666, 197.0686, 140.0985]) Cost: 1.619770
Epoch 20/20 hypothesis: tensor([152.8020, 183.6731, 180.9677, 197.0699, 140.1000]) Cost: 1.619033
```

Final (y)
152
185
180
196
142

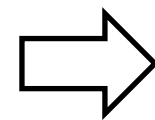
- Cost Gradually Decreases
- $H(x)$ gradually gets closer to y .
- Depending on the learning rate, it can also diverge.

Multi-Variable Linear Regression

- Streamlining the Code with nn.Module
 - In a class that inherits from nn.Module, all parameters automatically have their gradients computed.

```
# Model Init
W = torch.zeros((3, 1), requires_grad=True)
b = torch.zeros(1, requires_grad=True)

# H(x)
hypothesis = x_train.matmul(W) + b # or .mm or @
```



```
import torch.nn as nn

class MultivariateLinearRegressionModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(3, 1)

    def forward(self, x):
        return self.linear(x)

hypothesis = model(x_train)
```

- Create a Model by Inheriting from nn.Module
 - nn.Linear(3, 1)
 - Dim. of Input: 3
 - Dim. of Output: 1
- Put the hypothesis calculation in forward().
- PyTorch handles gradient calculations for you in backward().

Multi-Variable Linear Regression

- Streamlining the Code with F.mse_loss
 - Is there already a built-in loss function?

```
# Computation of cost  
cost = torch.mean((hypothesis - y_train) ** 2)
```

```
import torch.nn.functional as F  
  
# cost  
cost = F.mse_loss(prediction, y_train)
```

- We can use the loss functions provided by torch.nn.functional.
- It's easy to switch to other loss functions (e.g., l1_loss, smooth_l1_loss, etc.).

Multi-Variable Linear Regression

- Codes

```
# data
x_train = torch.FloatTensor([[73, 80, 75],
                             [93, 88, 93],
                             [89, 91, 90],
                             [96, 98, 100],
                             [73, 66, 70]])
y_train = torch.FloatTensor([[152], [185], [180], [196], [142]])

# Model Init
W = torch.zeros((3, 1), requires_grad=True)
b = torch.zeros(1, requires_grad=True)
model = MultivariateLinearRegressionModel()

# optimizer
optimizer = optim.SGD([W, b], lr=1e-5)
```

```
import torch.nn as nn

class MultivariateLinearRegressionModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(3, 1)

    def forward(self, x):
        return self.linear(x)
```

```
nb_epochs = 20
for epoch in range(nb_epochs + 1):

    #  $H(x)$ 
    hypothesis = x_train.matmul(W) + b # or .mm or @
    Hypothesis = model(x_train)

    # cost
    cost = torch.mean((hypothesis - y_train) ** 2)
    cost = F.mse_loss(prediction, y_train)

    #  $cost \propto H(x)$ 
    optimizer.zero_grad()
    cost.backward()
    optimizer.step()

    print('Epoch {:4d}/{:4d} hypothesis: {} Cost:
          {:.6f}'.format( epoch, nb_epochs,
                          hypothesis.squeeze().detach(), cost.item()))
```

인생은 실전 (Assignment)

- Let's Build an Automatic Writing Evaluation (AWE) System
 - 1) Read the data from **AWE_1000_score.csv**
 - 2) Count how many sentences are in the content column, and store that count in a new sentence column.
 - 3) Use the number of sentences as input to a model that predicts the writing evaluation score of each entry.
 - 4) Use both the number of sentences and the number of phrases as input.

(But how should we count the number of phrases? It's up to you!! There is no correct answer)

	id	nationality	sex	title	content	phrase	sentence
0	A100000_v01.txt_1.txt	중국	남자	사진기 빌리기	하지만 빌 리씨하고 나오코씨 는 모두 사 진기가 없 었어요. 그 래서 빌리 씨는 모하 메드 씨...	1.0	5.0

04. Data batch Processing

The Big Input Problem

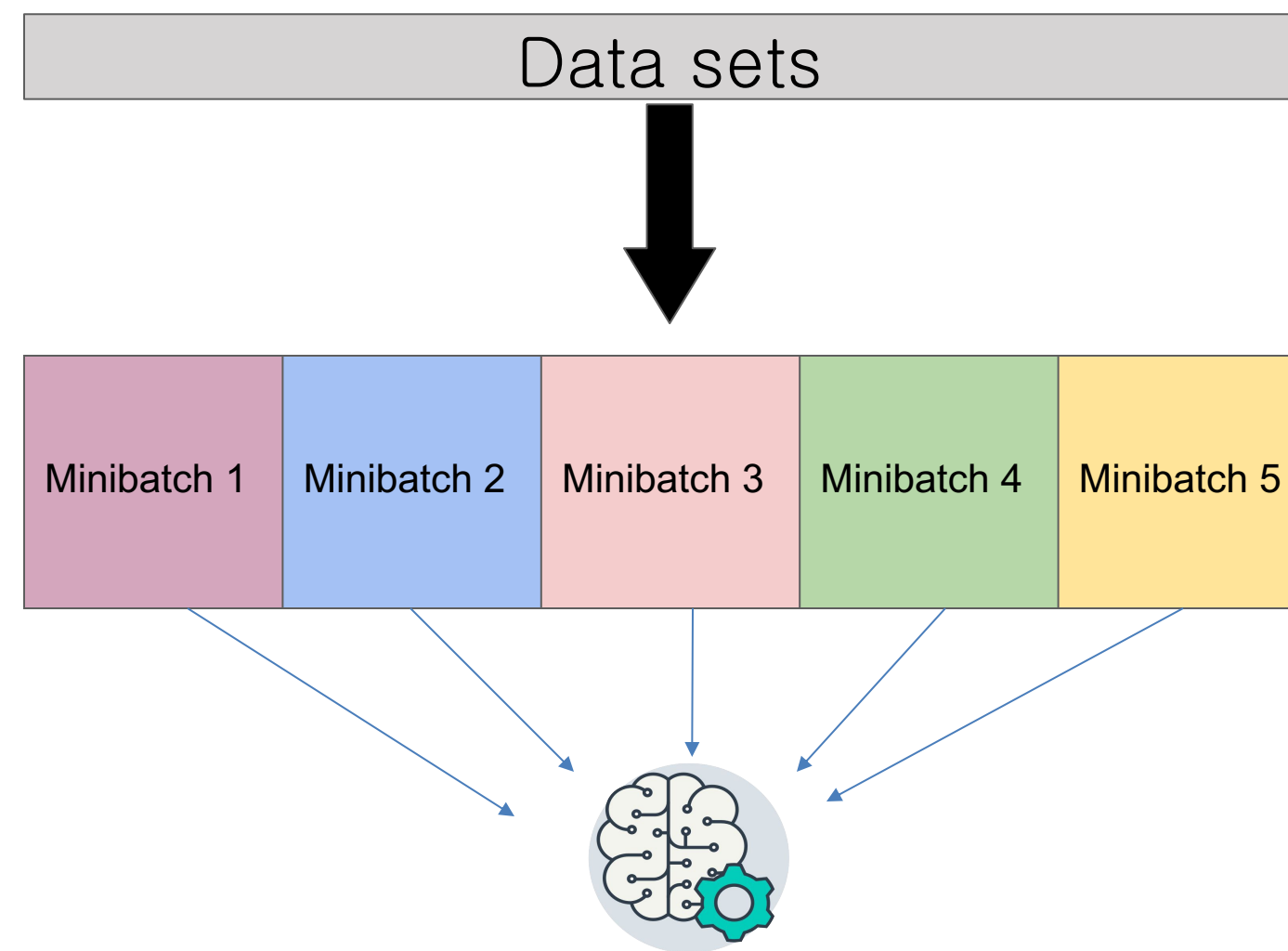
- Consider building an Automatic Writing Evaluation (AWE) system.
 - Do we really want to process all 1,000 training data points at once, compute the loss, and then update ***W***?
 - What if we had 1,000,000 training examples?
 - We'd spend ages just doing one update. We'd have to rely on parallel operations, and even that might be unmanageable.

$$y = \underset{\text{Weight}}{W}x + \underset{\text{Bias}}{b}$$

Let assume that we got X with the dimension of [1000000x2].. Can our machine handle it?

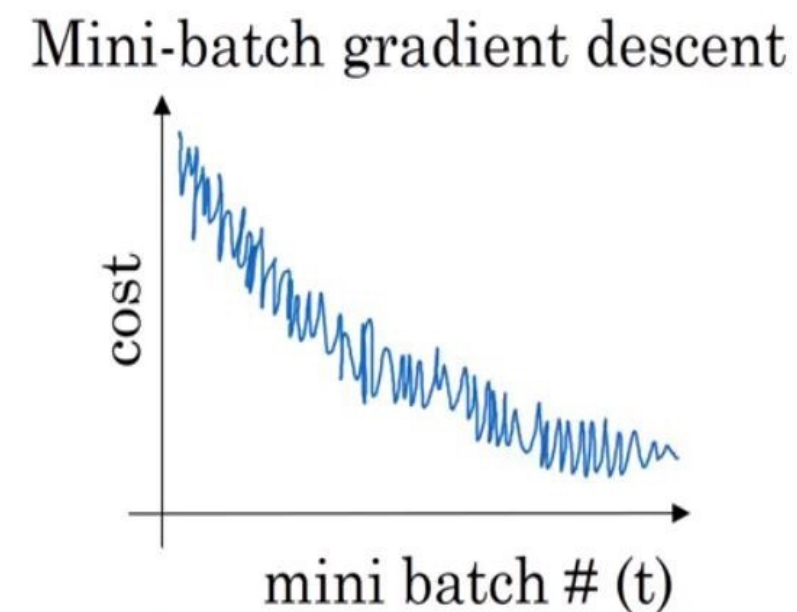
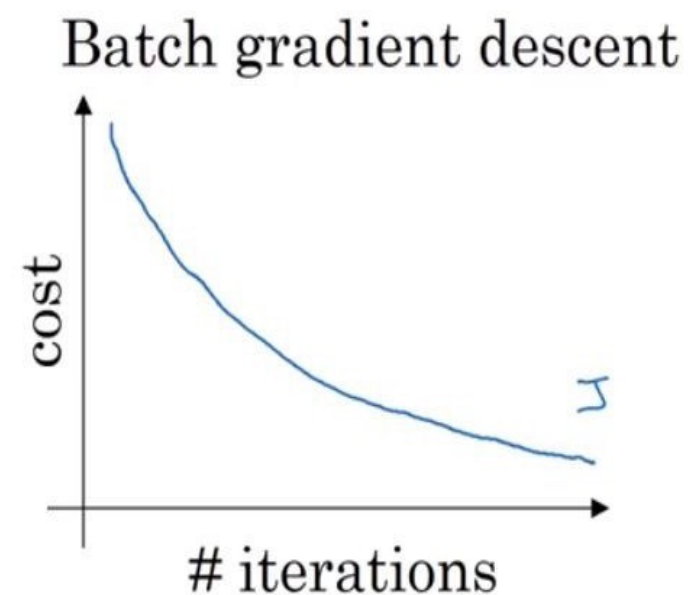
Solution of The Big Input

- Minibatch Gradient Descent: Train on Subsets of the Data



Solution of The Big Input

- Minibatch Gradient Descent: Train on Subsets of the Data
 - Updates can be performed more quickly.
 - Because we're not using all the data, there's a risk of updating in the wrong direction.



Minibatch on Pytorch

- Define a **Dataset** for Batch Processing

```
from torch.utils.data import Dataset

class CustomDataset(Dataset):
    def __init__(self):
        self.x_data = [[73, 80, 75],
                        [93, 88, 93],
                        [89, 91, 90],
                        [96, 98, 100],
                        [73, 66, 70]]
        self.y_data = [[152], [185], [180], [196], [142]]

    def __len__(self):
        return len(self.x_data)

    def __getitem__(self, idx):
        x = torch.FloatTensor(self.x_data[idx])
        y = torch.FloatTensor(self.y_data[idx])

        return x, y

dataset = CustomDataset()
```

- Inherit from torch.utils.data.Dataset.
- `__len__()`: Returns the total number of data points in the dataset.
- `__getitem__(idx)`: When given an index idx, returns the corresponding input-output pair.

Minibatch on Pytorch

- Use a ***DataLoader*** to Retrieve Data Efficiently

```
from torch.utils.data import DataLoader

dataloader = DataLoader( dataset,
    batch_size=2, shuffle=True,
)
```

- Use `torch.utils.data.DataLoader`
- Typically set to a power of 2 (16, 32, 64, 128, 256, 512, etc.).
- `shuffle=True`: Shuffles the dataset each epoch, changing the order in which data is learned.

Minibatch on Pytorch

- Let's Start Training with the Prepared Data

```
nb_epochs = 20
for epoch in range(nb_epochs + 1):
    for batch_idx, samples in enumerate(dataloader):
        x_train, y_train = samples
        #  $H(x)$ 
        prediction = model(x_train)

        # cost
        cost = F.mse_loss(prediction, y_train)

        #  $cost \propto H(x)$ 
        optimizer.zero_grad() cost.backward()
        optimizer.step()

    print('Epoch {:4d}/{:4d} Batch {}/{} Cost: {:.6f}'.format(
        epoch, nb_epochs, batch_idx+1, len(dataloader),
        cost.item()
    ))
```

- `enumerate(dataloader)`: Yields the minibatch index and its data.

- `len(dataloader)`: The number of minibatches in one epoch.

Do it your self! (Not an assignment)

- Let's Build an Automatic Writing Evaluation (AWE) System!
 - **1)** Read the file AWE_1000_score.csv.
 - **2)** Implement a Dataloader that loads the content column in batches of size 4.
 - **3)** Make sure that the Dataloader output is converted into a TF-IDF (DTM) representation of each batch of sentences.

Do it your self! (Not an assignment)

- Let's Build an Automatic Writing Evaluation (AWE) System Again!
 - 1) Read the file AWE_1000_score.csv.
 - 2) Implement a model that uses the content column as input to predict the writing score.
- (Advanced) For Korean writing evaluation, what additional features might help?
 - For example, the use of many difficult words, or leveraging word2vec?

```
X_train[1234]
```

'광고 한국 아무개 bank 대출 한도 금리 자신
대출 50 금융 신분 월급 절반 정도 카드 으로'

```
vectorizer=TfidfVectorizer(ngram_range=(1, 3), min_df=2, max_features=10000, sublinear_tf=True, lowercase=False, use_idf=True)  
X_train_vec= vectorizer.fit_transform(X_train)  
X_test_vec= vectorizer.transform(X_test)
```

Thank you.